

1. What is React, and why is it used in web development?

React is an open-source JavaScript library developed by Facebook for building user interfaces, particularly for single-page applications. It is widely used in web development because of its component-based architecture, which allows developers to build reusable UI elements, making code easier to maintain and scale. Additionally, its use of the Virtual DOM ensures fast and efficient rendering.

2. What is JSX? Mention one advantage of using JSX.

JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML-like code directly within JavaScript files.

Advantage: It makes the code much easier to read and write by visually structuring the UI components in a way that resembles standard HTML, while still harnessing the full programmatic power of JavaScript.

3. Explain the difference between Props and State in React.

Props (Properties): These are read-only components used to pass data from a parent component down to a child component. They are immutable and cannot be modified by the receiving component.

State: This is an internal data store specific to a component. It is mutable and can be updated by the component itself (usually via user interaction or network responses), which triggers a re-render of that component.

4. What is the purpose of the useState() hook? Give a simple example.

The `useState()` hook allows developers to add and track state within functional components. It returns an array containing the current state value and a function to update it.

```
import { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (
```

```

<div>

  <p>You clicked {count} times</p>

  <button onClick={() => setCount(count + 1)}>Click me</button>

</div>

);

}

```

5. What is the `useEffect()` hook, and when is it commonly used?

The `useEffect()` hook lets you perform side effects in functional components. It runs after the component renders. It is commonly used for tasks that interact with the outside world, such as fetching data from an API, setting up subscriptions, or manually manipulating the DOM.

6. What is the Virtual DOM? How does it improve React application performance?

The Virtual DOM is a lightweight, in-memory representation of the actual browser DOM. When a component's state or props change, React updates the Virtual DOM first. It then compares (diffs) the updated Virtual DOM with a snapshot of the previous version to calculate the exact, minimal changes needed. It then applies only those specific changes to the real DOM, significantly reducing expensive browser layout recalculations and improving performance.

7. Why is the `key` prop important when rendering lists in React?

The `key` prop gives React a unique identifier for each element in an array. This helps React efficiently determine exactly which items have been added, changed, or removed during re-renders, preventing the need to tear down and rebuild the entire list.

8. Explain the purpose of the `useRef()` hook with an example use case.

The `useRef()` hook is used to persist values across renders without causing the component to re-render when the value changes.

-

Example Use Case: It is frequently used to directly access and manipulate a DOM element, such as automatically setting focus on an input field when a component mounts.

-

```
import { useRef, useEffect } from 'react';

function TextInputWithFocusButton() {

  const inputEl = useRef(null);

  useEffect(() => {

    // Focus the input element automatically when the component mounts

    inputEl.current.focus();

  }, []

);

  return <input ref={inputEl} type="text" />;
}
```

9. What is the difference between useMemo() and useCallback()?

-

useMemo(): Memoizes and returns a **computed value**. It is used to optimize performance by caching the result of an expensive calculation so it doesn't need to be recalculated on every render unless its dependencies change.

-
-

useCallback(): Memoizes and returns a **function definition**. It is used to prevent child components from unnecessarily re-rendering by ensuring that the reference to a callback function remains identical across renders, as long as its dependencies haven't changed.

-

10. What is the Context API, and when would you use it instead of passing props through multiple components?

The Context API is a built-in React feature that allows you to share state globally across the entire component tree without having to manually pass props down through every single nested level (a problem known as "prop drilling"). You would use it for global data that many different components need access to, such as the user's authentication status, the preferred UI theme (light/dark mode), or localized language preferences.

